



PDF Download
3706628.3708868.pdf
24 December 2025
Total Citations: 1
Total Downloads: 1183

DL Latest updates: <https://dl.acm.org/doi/10.1145/3706628.3708868>

RESEARCH-ARTICLE

OLA: An FPGA-based Overlay Accelerator for Privacy Preserving Machine Learning with Homomorphic Encryption

YANG YANG, University of Southern California, Los Angeles, CA, United States

RAJGOPAL KANNAN, U.S. Army Research Laboratory, Adelphi, MD, United States

VIKTOR K PRASANNA, University of Southern California, Los Angeles, CA, United States

Open Access Support provided by:

U.S. Army Research Laboratory

University of Southern California

Published: 27 February 2025

Citation in BibTeX format

FPGA '25: The 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays
February 27 - March 1, 2025
CA, Monterey, USA

Conference Sponsors:
SIGDA

OLA: An FPGA-based Overlay Accelerator for Privacy Preserving Machine Learning with Homomorphic Encryption

Yang Yang
yyang172@usc.edu
University of Southern California
Los Angeles, California, USA

Rajgopal Kannan
rajgopal.kannan.civ@army.mil
DEVCOM Army Research Office
Los Angeles, California, USA

Viktor K. Prasanna
prasanna@usc.edu
University of Southern California
Los Angeles, California, USA

Abstract

Homomorphic Encryption (HE) is a promising technique for protecting user privacy in cloud-based Machine Learning (ML) inference. However, homomorphically encrypted operations are orders of magnitude slower than the corresponding unencrypted operations due to high computation and memory bandwidth requirements. FPGAs are attractive platforms for designing domain-specific accelerators. Manually programming FPGAs for HE ML inference is challenging, as different HE parameters and operations require different mapping strategies. We propose OLA, the first FPGA-based overlay accelerator for low latency computation on homomorphically encrypted data. OLA eliminates the need for manual and time consuming FPGA programming by providing a Python-based interface and a compiler that efficiently maps OLA programs to hardware instructions for execution. The hardware architecture and instruction set are co-designed to accelerate common HE primitives, while the compiler maps all the required HE operations to these primitives for efficient execution. OLA features two optimizations to address memory bandwidth challenges in HE computation. First, we propose an asynchronous dataflow execution model where the compiler manages data processing order and the architecture enforces it at run time. This approach enables guaranteed data reuse via on-chip SRAMs. Second, we design latency-aware instruction scheduling in the compiler to reduce data reuse distance. We implement the overlay accelerator on the AMD Alveo U280 FPGA. We evaluate the effectiveness of the proposed overlay accelerator by executing various HE operations, HE linear algebra benchmarks, and end-to-end ML inference. Experimental results show that OLA reduces the latency of HE operations by up to $683\times$ ($4.4\times$) compared with State-Of-The-Art (SOTA) CPU (GPU) implementations. It achieves up to $3212\times$ speedup in latency compared with the SOTA CPU implementations for HE ML inference.

CCS Concepts

• **Computer systems organization** → **Parallel architectures**; • **Security and privacy** → **Security in hardware**.

Keywords

Homomorphic Encryption, FPGA Acceleration, Parallel Computing

ACM Reference Format:

Yang Yang, Rajgopal Kannan, and Viktor K. Prasanna. 2025. OLA: An FPGA-based Overlay Accelerator for Privacy Preserving Machine Learning with Homomorphic Encryption. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25)*, February 27–March 1, 2025, Monterey, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3706628.3708868>

1 Introduction

Homomorphic Encryption (HE) provides a practical way for privacy preserving computations. Existing studies have demonstrated that the operations supported by HE can effectively perform a wide range of machine learning inference tasks [5, 26, 31, 34, 37, 39, 40]. Low latency HE ML inference is crucial for many real-world applications. In HE, the client provides encrypted data to a cloud server, on which the inference is performed *without* decrypting the data [18]. While HE offers strong privacy guarantees in cloud-based ML inference [4], it comes at a high cost: computation using HE is orders of magnitude slower than performing the same computation on unencrypted data. All HE computations take place within a polynomial ring with a large modulus, typically reaching thousands of bits [4]. In addition, most of the HE operations use modular arithmetic, which is costly to implement in hardware. The high degree polynomial modular arithmetic and the large coefficients not only increase the computational complexity, but also incur significant overheads on off-chip data transfers [15]. These challenges highlight the need for domain-specific accelerators tailored for HE computations [22].

With the fine-grained programmable architecture and tight integration with High Bandwidth Memory (HBM), FPGAs are an ideal platform for accelerating data intensive applications such as HE-based computation. However, it is non-trivial to develop FPGA accelerators to efficiently perform computations on homomorphically encrypted data. First, the design space for HE hardware acceleration is large [49, 51], and HE applications [26, 30, 31] use a wide range of parameters and require different combinations of HE operations. Developing FPGA accelerators for each HE application demands significant effort. Second, programming FPGAs using Hardware Description Language (HDL) or High-Level Synthesis (HLS) requires extensive hardware knowledge and involves a long development cycle [12, 38]. Although several designs have been proposed to operate with fixed HE parameters [2, 41, 53], adapting these designs to different parameters requires time-consuming HDL programming or HLS implementation tuning. Previous FPGA designs and frameworks [49, 53, 54] have primarily focused on hardware implementations for specific HE operations, overlooking the software aspects of the workflow, such as how to program the FPGA accelerators for a given HE application.



This work is licensed under a Creative Commons Attribution International 4.0 License.

FPGA '25, February 27–March 1, 2025, Monterey, CA, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1396-5/25/02
<https://doi.org/10.1145/3706628.3708868>

In this paper, we propose *OLA*, an FPGA-based overlay accelerator designed for privacy preserving Machine Learning (ML) using HE. *OLA* incorporates an Instruction Set Architecture (ISA) and a compiler, offering a flexible programming abstraction alongside an architecture specifically optimized for HE computations. The ISA enables the execution of HE primitives, while the compiler takes as input an HE program written in a Python-based Domain-Specific Language (DSL) and generates a sequence of instructions for the overlay accelerator to execute. *OLA* incorporates optimizations across the architecture, ISA and compiler to reduce HE ML inference latency. To the best of our knowledge, this is the first work to develop an end-to-end FPGA-based overlay accelerator for HE that includes a software programmable instruction set architecture and compiler. The contributions of this paper are:

- We propose *OLA*, an FPGA-based programmable overlay accelerator for HE ML inference. *OLA* features a compiler that maps HE programs written in a Python-based DSL to instructions optimized for low latency execution on the accelerator.
- We identify a set of fundamental HE primitives common to HE operations. The hardware architecture is designed to accelerate these primitives, while the compiler maps all the HE operations onto these primitives. This approach offers reuse of FPGA compute resources (e.g., DSPs) across HE operations, thereby improving the computation utilization.
- *OLA* features two optimizations to reduce DRAM transfers and hide DRAM access latency.
 - We propose an asynchronous dataflow execution model that allows the compiler to fully manage the data processing order based on the dependencies of HE operations. We co-design the ISA and architecture to seamlessly support this execution model. This approach guarantees data reuse through on-chip SRAMs and significantly reduces DRAM transfers.
 - We develop DRAM latency-aware instruction scheduling policies in the compiler to overlap data transfers with computation and reduce data reuse distance. This optimization minimizes stall cycles caused by data access delays.
- We demonstrate the effectiveness of the overlay accelerator on the AMD Alveo U280 FPGA. We evaluate a range of HE operations and HE ML inference benchmarks across different HE parameters.
 - The proposed compiler takes less than 1 second to generate the instruction sequence for various HE CNN models, which is significantly faster than performing FPGA implementation flow for each model.
 - The proposed accelerator reduces latency for HE operations by up to 683× (4.4×) compared to State-Of-The-Art CPU (GPU) implementations.
 - Our design achieves up to 3212× speedup in latency compared to SOTA CPU designs for HE ML inference.

2 Background

2.1 Homomorphic Encryption (HE)

Our framework targets the CKKS scheme as it is best suited for machine learning tasks [10, 32]. The proposed techniques can be

applied to other HE schemes [6, 7]. We follow the notation used in [23, 49]. The CKKS parameters are detailed in Table 1. A cleartext vector m of $N/2$ fixed-point numbers is first encoded into a plaintext polynomial (pt). This pt is then encrypted into a pair of ciphertext polynomials (ct), $\llbracket m \rrbracket = (a, b)$, using a secret key s . Both pt and ct have a polynomial degree of $N - 1$. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$. The field of a fresh ciphertext is $\mathcal{R}_Q^2 = (\mathcal{R}/Q\mathcal{R})^2$, which consists of a pair of polynomials with coefficients from $\mathbb{Z}_Q$, i.e., integers modulo Q . The modulus Q is chosen to be sufficiently large (hundreds or even thousands of bits) to prevent data corruption during encrypted computation. The Residue Number System (RNS) is used to manage this complexity [10]. Specifically, let $Q = \prod_{i=0}^L q_i$ be the modulus of a ciphertext with a computational level L . A polynomial in $\mathcal{R}_Q$ has $L + 1$ limbs, with the coefficients of the i -th limb belonging to $\mathbb{Z}_{q_i}$. HE operations increase the encryption noise and reduce the level of the ciphertext. When the level reaches zero, decryption is no longer possible. Bootstrapping can reset the noise, enabling Fully Homomorphic Encryption (FHE). The security level (λ) of the CKKS scheme depends on the polynomial degree N and the ciphertext modulus Q .

Table 1: CKKS Parameters [10]

Parameter	Description
N	Degree of the plaintext and ciphertext polynomials.
L	Maximum computational level of a ciphertext.
l	Current level of a ciphertext, $0 \leq l \leq L$. A ciphertext with level l has $l + 1$ limbs.
Q	Maximum modulus of a ciphertext coefficient. It is represented by $L + 1$ co-primes q_i . $Q = \prod_{i=0}^L q_i$.
P	Product of k additional co-primes p_i for the raised modulus. $P = \prod_{i=0}^{k-1} p_i$
dnum	Number of digits in the switching key.
α	Number of limbs of a single digit during key switching decomposition. $\alpha = \lceil (L + 1)/\text{dnum} \rceil$.
β	An l -limb ciphertext is split into β digits during key switching decomposition, where $\beta = \lceil (l + 1)/\alpha \rceil$.

2.2 HE Operations

CKKS supports the following operations.

- **PtCtAdd** and **PtCtSub** perform element-wise addition and subtraction between a plaintext m_0 and a ciphertext $\llbracket m_1 \rrbracket$ respectively.
- **CtCtAdd** adds two encrypted vectors $\llbracket m_0 \rrbracket$, $\llbracket m_1 \rrbracket$ and outputs the element-wise sum of the two vectors, $\llbracket m_0 + m_1 \rrbracket$. **CtCtSub** performs element-wise subtraction and produces $\llbracket m_0 - m_1 \rrbracket$.
- **PtCtMult** multiplies a plaintext m_0 with a ciphertext $\llbracket m_1 \rrbracket$ and outputs $\llbracket m_0 \cdot m_1 \rrbracket$, where $\cdot$ denotes element-wise multiplication of the two vectors.
- **CtCtMult** performs element-wise multiplication between two ciphertext $\llbracket m_0 \rrbracket$ and $\llbracket m_1 \rrbracket$ and outputs $\llbracket m_0 \cdot m_1 \rrbracket$.
- Let $\llbracket m \rrbracket$ be a ciphertext of vector $(c_0, c_1, c_2, \dots, c_n)$, where $n = N/2$. **Rotate** by r slots outputs $\llbracket \phi_r(m) \rrbracket = (c_r, c_{r+1}, \dots, c_{n-1},$

$c_0, c_1, \dots, c_{r-1}$), i.e., elements of vector are circularly shifted by r slots. Rotate is not a simple coefficient shift of ciphertext polynomials.

- The **Rescale** operation adjusts the scaling factors of the ciphertext, to prevent noise overflow [10].
- The **Bootstrap** operation resets the level of a ciphertext to its initial level L . In this paper, we use the bootstrapping algorithm described in [20].

CtCtMult multiplies the limbs of two input ciphertext (LimbMult). The result of LimbMult is encrypted with secret key s^2 . KeySwitch changes the encryption key back to s homomorphically. The complete algorithm is described in Algorithm 1. The first subroutine of KeySwitch is Decomp (Line 4), where the l limbs are divided into β digits (groups), each containing α limbs. Next, the ModUp subroutine expands each group to $\alpha\beta + k - 1$ limbs. The key switching key ($\text{ksk}_{s^2 \rightarrow s}$) consists of a matrix of polynomials with dimensions $2 \times \beta$, where each matrix element contains $\alpha\beta + k - 1$ limbs. The InnerProd subroutine performs matrix-vector multiplication of limbs between an input vector of size β limbs and the $\text{ksk}_{s^2 \rightarrow s}$. This is followed by the ModDown subroutine, which reduces the limb count of the InnerProd output from $\alpha\beta + k - 1$ to $l + 1$ limbs. Finally, a modular addition of limbs (LimbAdd) produces the output ciphertext (Line 8). The subroutines Decomp, ModUp, InnerProd and ModDown are described in Algorithm 3, 2, 10, and 4 respectively in [23]. Ciphertext limbs are in the evaluation domain through the Number Theoretic Transform (NTT). The BaseConv operation within ModUp and ModDown requires the ciphertext be in polynomial form [10]. Therefore, INTT and NTT are frequently used across various subroutines [23]. The first step of Rotate is the Automorph subroutine, which involves permuting the coefficients of each limb using the mapping $i \mapsto \sigma_r(i)$, where $\sigma_r(i) = i \cdot 5^r \bmod N$ for $0 \leq i \leq N - 1$. The output ciphertext of Automorph requires the same KeySwitch subroutine as in CtCtMult to obtain a valid output. The Rescale operation can be considered as a special implementation of ModDown subroutine in which the number of limbs is reduced by one (i.e., from l to $l - 1$). For a more formal description of various subroutines, we refer readers to [10].

Algorithm 1 CtCtMult($\llbracket m_0 \rrbracket, \llbracket m_1 \rrbracket, \text{ksk}_{s^2 \rightarrow s}$)

```

1:  $(a_0, b_0) \leftarrow \llbracket m_0 \rrbracket$ 
2:  $(a_1, b_1) \leftarrow \llbracket m_1 \rrbracket$ 
3:  $(a_2, b_2, c_2) \leftarrow (a_0 a_1, a_0 b_1 + a_1 b_0, b_0 b_1)$            ▶ LimbMult
4:  $a_2 \leftarrow \text{Decomp}_\beta(a_2)$ 
5:  $\hat{a}_2 \leftarrow \text{ModUp}(a_2)$ 
6:  $x, y \leftarrow \text{InnerProd}(\text{ksk}_{s^2 \rightarrow s}, \hat{a}_2)$ 
7:  $\hat{x}, \hat{y} \leftarrow \text{ModDown}(x), \text{ModDown}(y)$ 
8: return  $(b_2 + \hat{x}, c_2 + \hat{y})$                                        ▶ LimbAdd
```

3 Overview Of OLA

3.1 System Overview

Target Applications: This work focuses on machine learning inference in various HE-based applications [16, 17, 24, 25]. The input and output data for the ML model are homomorphically encrypted, and all computations are performed using HE operations.

Python DSL: Users define the desired HE computations and HE parameters for a given HE ML inference using a Python-based

Domain Specific Language (DSL). The DSL has built-in data types and APIs to support various HE operations (Section 6).

Compiler: The inputs to the compiler is a program written in the Python-based DSL. The front-end of the compiler transforms the input program into a *computation graph*, an Intermediate Representation (IR) of the program. It then applies DRAM related optimizations (Section 6.2) and generates a sequence of instructions for execution on the overlay accelerator.

Instruction Set: We develop a domain-specific instruction set for HE to serve as the bridge between the compiler and the overlay accelerator. The compiler translates the input program into this instruction set, which is then executed by the overlay accelerator.

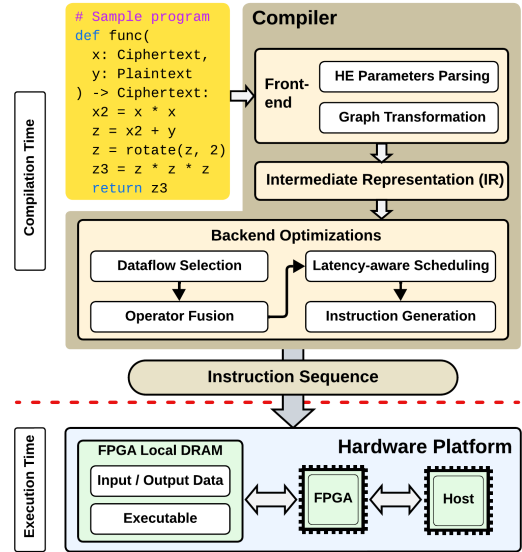


Figure 1: Overall workflow of OLA.

Hardware Platform: The hardware platform includes an FPGA device, FPGA local DRAM, and a host CPU. We deploy the overlay accelerator on the FPGA device, where it performs all HE computations. The FPGA local DRAM stores the encrypted input and output data, the ML model, and the binary executable generated by the compiler. The compiler is executed on the host CPU.

3.2 Workflow

Figure 1 illustrates the overall workflow of OLA. The compiler extracts the specifications of HE computations and HE parameters from the input program and creates a computation graph of HE operations. These operations are then transformed into HE subroutines, forming the Intermediate Representation (IR) of the input program. Once the IR is generated, the compiler backend performs a series of optimization steps, including dataflow selection, subroutine fusion, and instruction scheduling. Finally, the compiler generates a sequence of instructions in binary format that is sent to the overlay accelerator for execution. The overlay accelerator receives the binary instruction file (executable) along with the encrypted input data. It loads the executable into the control units and initiates execution. The executable completes once the last instruction is retired, and the generated output is stored in the FPGA DRAM. OLA supports any set of HE parameters defined by CKKS scheme. OLA compiler decomposes HE computations using

different HE parameters into HE-parameter-agnostic IR and ISA for execution on the overlay accelerator.

4 Instruction Set Design

4.1 Decomposition of HE Operations

We identify a fundamental set of HE primitives that are commonly shared across HE operations. The instruction set and hardware architecture are optimized to accelerate these HE primitives, while the compiler handles the task of translating all HE operations into these primitives. This approach enables the efficient reuse of compute resources in the FPGA across (e.g., DSPs) different HE operations.

HE Operations. We develop a Python-based Domain-Specific Language (DSL) to express HE computations using HE operations and their parameters. OLA supports all the HE operations described in Section 2.2. Previous studies have demonstrated that this set of HE operations can be used to perform a wide variety of HE-based machine learning inference tasks [5, 26, 30, 31, 37].

HE Subroutines. HE operations can be decomposed into various *HE subroutines*, such as Ntt, Intt, Automorph, Decomp, BaseConv, and KskInnerProd. An HE subroutine is an operation whose datapath is implemented using multiple HE primitives but is not directly exposed to the DSL. The primary purpose of representing HE operations as subroutines is to enable the compiler to apply optimizations that reduce latency (See Section 6.1).

HE Primitives. HE subroutines are translated into HE primitives, which form the core of our ISA. These HE primitives include ModAdd, ModSub, ModMult, Butterfly, and Permute. Permute enables permutations of limb coefficients, while other primitives handle modular arithmetic operations. These primitives are extensively reused across HE operations.

4.2 Instruction Set Architecture

OLA ISA follows a vector instruction set architecture, where instructions operate on vectors as operands, and the vector length is specified within the instruction. Instructions have a fixed length of 128 bits. We rely on the compiler to specify instruction-level dependencies and data processing order, simplifying the architecture by eliminating the need for a complex scoreboard. This approach also provides a predictable data access pattern, enabling the compiler to optimize data reuse. We use a counting semaphore mechanism [45] to ensure that instructions are executed in the correct order. Each instruction includes a Sync and a Wait field to guarantee that the execution conditions are met before the instruction is processed. Figure 2 shows the instruction set of the overlay accelerator, with three common fields across all instructions:

- **OpCode:** Specifies the type of instruction. Our ISA supports eight instructions as listed in Figure 2.
- **Sync:** Indicates the instruction trigger action. Upon completion, an instruction sends a trigger to one of the functional units specified by this field.
- **Wait:** Specifies the prerequisites for execution. An instruction is ready to execute once the number of triggers received for this instruction meets the value defined in this field.

OLA instruction set comprises the following instructions.

DmaRead	OpCode	Sync	Wait	Src	Dst	Bytes	Rsvd
DmaWrite	OpCode	Sync	Wait	Src	Dst	Bytes	Rsvd
CoefModAdd	OpCode	Sync	Wait	Src0	Src1	Dst	Length
CoefModSub	OpCode	Sync	Wait	Src0	Src1	Dst	Length
CoefModMult	OpCode	Sync	Wait	Src0	Src1	Dst	Length
Butterfly	OpCode	Sync	Wait	Src	Twiddle Factor	Dst	Length
Permute	OpCode	Sync	Wait	Src	Dst	Config	Length
Barrier	OpCode	Rsvd	Wait	Rsvd			

Figure 2: OLA instruction set.

- **DmaRead:** This instruction fetches 2^{Bytes} bytes from DRAM (specified by Src) to the scratchpad of the overlay accelerator (specified by Dst).
- **DmaWrite:** This instruction stores 2^{Bytes} bytes from the scratchpad of the overlay accelerator (specified by Src) to DRAM (specified by Dst).
- **CoefModAdd/CoefModSub/CoefModMult:** Let $\circ$ be one of the operations in $+$, $-$, $\times$. This instruction performs $Dst = (Src0 \circ Src1) \bmod q$ on a vector of Length elements, where q is from a register specified by ModReg.
- **Butterfly:** Performs the butterfly operation defined in Ntt and Intt. The twiddle factor address in the scratchpad is specified by the TwiddleFactor field.
- **Permute:** This instruction performs permutation on a vector of Length elements. The permutation pattern is specified by Config field, which is a pointer to the scratchpad memory that contains the control signals. The Id field specifies the Permute Unit responsible for executing the instruction.
- **Barrier:** Stall issuing the next instruction until the functional units specified by this instruction have become idle.

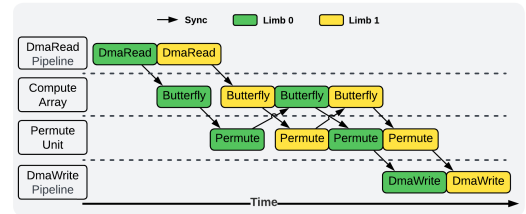


Figure 3: Execution of a two-stage NTT on two limbs.

4.3 Instruction Execution

The overlay accelerator issues instructions in order. We propose a trigger-based instruction execution mechanism to manage data dependencies, with dependency information explicitly defined in the ISA. Each instruction has an associated *trigger list*, with one bit assigned to each functional unit. When a trigger bit is set, the instruction increments a *trigger counter* for the corresponding functional unit upon completion. A dependent instruction is blocked from execution until the trigger counter value meets or exceeds the value specified in its Wait field. Figure 3 illustrates the execution process of a two-stage NTT on two limbs. The two NTTs are independent to each other and can be executed in parallel. The two DmaRead instructions are executed sequentially to fetch the NTT inputs, one for each limb. Upon completion of a DmaRead, the sync field signals that the Compute Array (Section 5.2), where the Butterfly instructions are executed, needs to be triggered. The instruction then updates the trigger counter of the Compute Array.

The Butterfly instruction is then activated and begins execution. Similarly, the Permute instruction for the same limb waits for the Butterfly instruction to complete. Finally, the DmaWrite instruction stores the results back to memory after the last stage of the NTT is completed. The trigger-based mechanism enables the compiler to control data processing order, improving data reuse through on-chip SRAM (Section 6.2).

5 Hardware Architecture

5.1 Architecture Overview

Figure 4 depicts the overall architecture. The *Instruction Buffer* stores the instructions to be executed. The architecture consists of four main functional units: *DMA Read*, *DMA Write*, *Compute Array*, and *Permute Units*. At the start of a program, instructions are fetched from DRAM to the Instruction Buffer. The *Instruction Decode and Dispatch* (IDD) unit dispatches the instructions to the corresponding functional units. Each functional unit has an instruction queue to store the issued instructions. Additionally, each unit contains a *Control* module to monitor instruction dependencies. Execution of the instruction at the head of the queue begins only when its trigger conditions are met. Polynomial limbs and twiddle factors are stored in the *Scratchpad Memory*. The Scratchpad Memory consists of 8 banks, each 512B wide. Each bank has one read port and one write port. The DMA Read unit executes the DmaRead instructions, fetching data from DRAM and storing it in the Scratchpad or Instruction Buffer. The DMA Write unit executes the DmaWrite instructions, reading data from the Scratchpad and storing it back in DRAM. The Compute Array handles all computation related instructions, while the Permute Units process the Permute instructions. The Barrier instruction is executed by the IDD unit, which pauses the issuing of subsequent instructions until the barrier conditions are met.

5.1.1 Design Considerations. We determine the throughput of the Compute Array based on the available DSP resources of the target FPGA. The Permute Units are configured to match the Ntt processing rate of the Compute Array. We configure the banking structure of the Scratchpad Memory to match the peak throughput of the Compute Array and Permute Units.

5.2 Compute Array

The Compute Array consists of an array of 512 *Compute Cells* that can be programmed to perform modular arithmetic. Figure 5 illustrates the architecture of the Compute Array. The Compute Array operates in a SIMD fashion – all Compute Cells execute the same instruction concurrently. The Control module within the Compute Array is responsible for reading instructions and providing control signals to all the Compute Cells. A set of 64 *Modulus Registers* is used to store the modulus (p_i and q_i in Table 1). Modulus is broadcast to all Compute Cells prior to computation. The Compute Cells support 32-bit arithmetic, which prior works have shown to be sufficient for HE operations [2, 43, 53]. For modular multiplication, we implement the Barrett reduction algorithm [21] based on a fully pipelined approach from Kim et al [29].

The Compute Array supports a special mode for executing 512 butterfly operations in parallel, commonly used in Ntt and Intt.

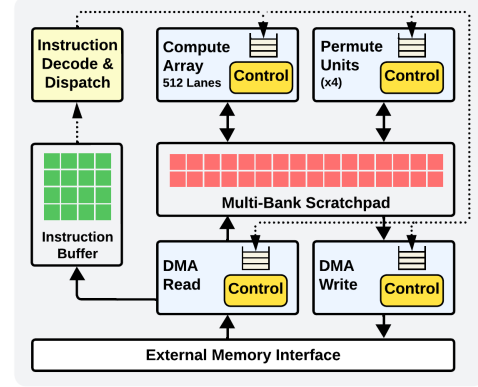


Figure 4: Overlay accelerator architecture.

Figure 5 illustrates the active datapath (highlighted by blue arrows) during the execution of a Butterfly instruction. In this mode, the modular multiplier is used to multiply a limb coefficient by a twiddle factor, and the result is then added to or subtracted from another limb coefficient. The operation produces two coefficients as outputs.

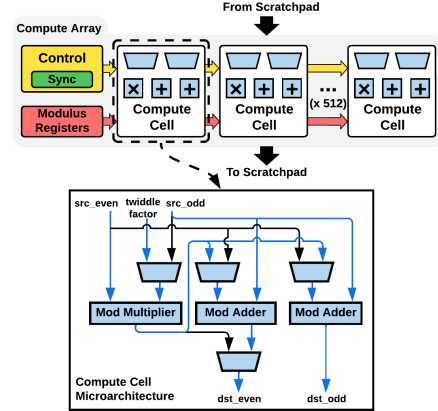


Figure 5: Architecture of the Compute Array.

5.3 Permute Units

The Permute Units enable permutations of limb coefficients, which is essential for Ntt, Intt and Automorph. The overlay accelerator includes four Permute Units, each capable of a permutation throughput of 128 coefficients per cycle. Due to the complex data access patterns, permutations in HE are difficult to process in parallel [53]. We design the Permute Unit using the Streaming Permutation Network (SPN) [9]. Figure 6 illustrates a Permute Unit with a throughput of $t = 4$ coefficients per cycle. It consists of three sub-networks: two spatial permutation networks based on the Butterfly topology and one temporal permutation network. Each spatial permutation network has $\log t$ stages, using 2×2 switches recursively to create a t -to- t connection. The spatial permutation networks do not have buffers internally, receiving t coefficients per cycle and directly performing permutations across these t coefficients before sending them to the next stage. The temporal permutation network has t reorder buffers. The first Spatial Permutation Network writes coefficients to the buffers in-order, while the second Spatial Permutation Network reads coefficients from the buffers out-of-order based on the permutation to be realized.

5.3.1 Programming the Permute Unit. A key limitation of the original SPN design [9] is the lack of a proper software programming interface. All control logic is hard-wired, making it impractical to support various permutation patterns required by Ntt and Automorph at runtime. To address this, we use the compiler to generate the control signals for the Permute Units and add ISA support that loads control signals at runtime. Each permutation pattern requires a specific set of control signals, which includes routing tables for spatial permutations and buffer access addresses for temporal permutations. The routing table contains control bits for the switches, with each switch having a single control bit per cycle. For the permutation of a vector of size N , each Permute Unit contains $t \log t$ switches, with each switch requiring N/t bits. Therefore the total control bits for spatial permutation is $N \log t$ bits. For Ntt and Intt operations, $\log N$ sets of control signals are needed, corresponding to the $\log N$ stages. For Automorph, one set of control signals is required per rotation distance. We add a Control Unit with three command queues: one for each of the two Spatial Permutation Networks and one for the Temporal Permutation Network. The compiler generates all control signals based on the input program. The DmaRead instruction fetches the control signals from DRAM into the scratchpad before executing a Permute instruction (ensured through instruction trigger list).

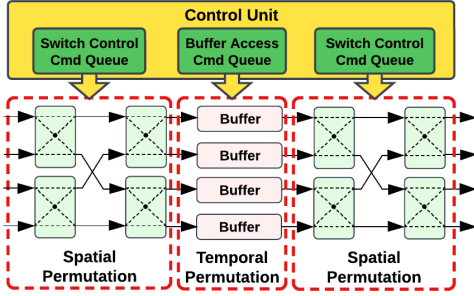


Figure 6: Architecture of the Permute Unit.

6 Compiler

The compiler takes a program written in Python-based DSL as input. The DSL provides the following built-in data types and operation APIs.

- Data types: Scalar, Plaintext and Ciphertext.
- Operation APIs: PtCtAdd, CtCtAdd, PtCtSub, CtCtSub, PtCtMult, CtCtMult, Rotate, Rescale, and Bootstrap

Users write HE programs using these custom data types and operation APIs. Listing 1 shows a simple matrix-vector multiplication example (matrix is packed in diagonal manner [24]). The encryption parameters, as detailed in Table 1, are specified via the HeParameters class. The compiler then translates the program into the hardware instructions for execution.

```

1 def matrix_vector_mult(
2     params: HeParameters,
3     A: List[Ciphertext],
4     x: Ciphertext,
5 ):
6     n: int = len(A)
7     result: ciphertext = Ciphertext(params)
8     for i in range(n):

```

```

9         result += rotate_left(x, i) * A
10    return result

```

Listing 1: Example program for matrix-vector multiplication using the proposed Python DSL.

Figure 7 provides an overview of the proposed compiler. The compilation process is performed in two main stages: i) the transformation stage; and ii) the optimization stage. In the transformation stage, we apply *HE Parameter Parsing*, which extracts the specifications of the HE program and parameters. Next, the compiler carries out *Operation Graph Transformation*, converting the Python program into a *computation graph*. Each node in the graph represents an HE operation, while the edges indicate dependencies between operations and the flow of ciphertext. The compiler then performs *Operation Lowering to IR*, transforming the computation graph to an *IR graph* for further optimizations. During the optimization stage, the compiler applies various optimizations (See Section 6.2), and generates an instruction sequence for execution.

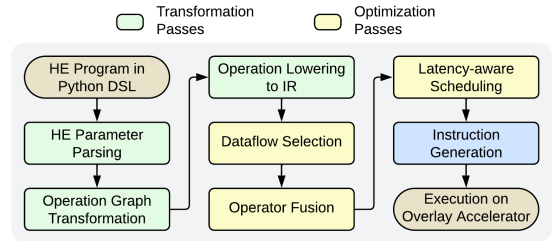


Figure 7: Code generation flow of the proposed compiler.

6.1 Intermediate Representation (IR)

We define HE-specific types and operators within the IR, laying the foundation for various compiler optimizations before instruction generation. Table 2 lists the types and operators in the IR.

Table 2: Types and operators in the IR used by the compiler.

Types		
LimbVector	LimbMatrix	Scalar
Operators		
LimbElementwise	Ntt/Intt	KskInnerProd
Automorph	Decomp	BaseConv
Merge	Split	

Types: The IR uses LimbVector and LimbMatrix to represent Plaintext and Ciphertext. A LimbVector is a vector of limbs, while a LimbMatrix is a matrix of limbs. A Plaintext is a LimbVector of length l , where l is the number of moduli in the Plaintext. A Ciphertext is a LimbMatrix of shape $2 \times l$, with each element representing a limb of degree $N - 1$. Both types also include attributes such as the modulus for each limb. We use Scalar type to represent function arguments of HE operations.

Operators: The operators in the IR are defined based on the HE subroutines [23, 49]. A brief summary is provided below.

- **Computation:** LimbElementwise performs addition, subtraction, and multiplication between two limb vectors. Ntt and Intt executes the Ntt and Intt transformations on a limb vector. KskInnerProd carries out the key switching inner product operation between two limb matrices. Decomp converts

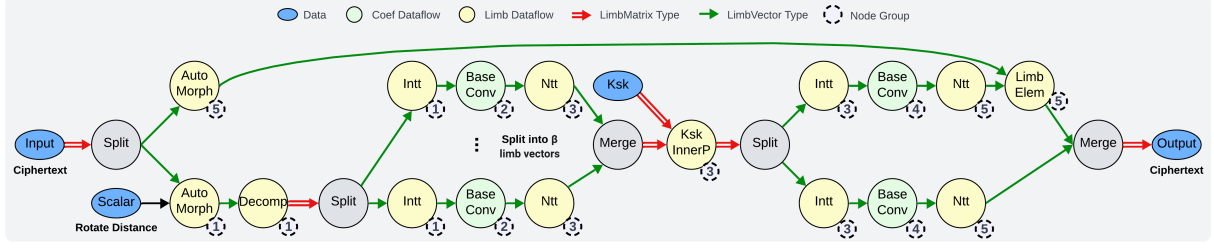


Figure 8: Lowering Rotate operation to IR.

a limb vector into a limb matrix through decomposition. BaseConv applies the Chinese Remainder Theorem to increase or decrease the length of limb vectors.

- Conversion: Split splits a limb matrix into an array of limb vectors. Merge combines an array of limb vectors into a limb matrix. Both operators are not materialized.

The Operation Lowering to IR pass first converts the node’s inputs and outputs to IR types, then rewrites the node according to the HE subroutines needed to perform the operation. Figure 8 shows the IR graph of Rotate operation.

6.2 Compiler Optimizations

The main challenge in efficiently computing on homomorphically encrypted data is the long DRAM latency and large amount of data transfers [15]. Optimizing the data transfer overheads while maintaining flexibility is difficult, as manually tuned data processing flows that reduce unnecessary transfers can limit programmability. To tackle this, OLA implements two optimizations in the compiler, leveraging support from the ISA and hardware architecture.

6.2.1 Asynchronous Dataflow Execution Model. We propose an asynchronous dataflow execution model that rearranges the data processing order of ciphertext limbs to improve data reuse through on-chip SRAM. The execution model leverages two key features of OLA: i) the trigger-based instruction execution mechanism which ensures a fixed Scratchpad Memory access pattern, and ii) the compiler’s full visibility of the computation graph, which provides a holistic view of data reuse opportunities. The goal is to perform as many operations as possible once the data is loaded into the scratchpad memory. This is achieved through two compiler optimization passes: *Dataflow Selection* and *Operator Fusion*. First, we classify each node in the IR graph into two categories based on computation patterns: the *Limb Pattern* and the *Coefficient Pattern* [15, 52]. In the Limb Pattern, the i -th output limb is calculated from the corresponding limb in the input ciphertext. In contrast, the Coefficient Pattern computes the i -th coefficient of an output limb by using the i -th coefficient from all the input limbs. IR operators that follow the Limb Pattern include LimbElementwise, Automorph, Ntt/Intt, Decomp, and KskInnerProd, while BaseConv uses the Coefficient Pattern. We adopt the dataflows from [49], assigning nodes with a Limb Pattern to the *Limb Dataflow* and those with a Coefficient Pattern to the *Coefficient Dataflow*. In the Limb Dataflow, data is processed at the limb level, with the functional units processing on one limb at a time before working on the next. In the Coefficient Dataflow, the first set of coefficients of all the output limbs are computed, followed by the next set of coefficients. When two

adjacent nodes both use the Limb Dataflow, their processing can be fused, avoiding the need to write intermediate results to DRAM. However, prior work [49] has two limitations: (i) fusion is limited to HE subroutines with a single limb vector, and (ii) the fusion rule is hard-coded, fusion is limited to subroutines within the same HE operation. Additionally, previous works do not provide a fusion interface that is easily accessible during compilation. These limitations restrict the number of nodes that can be fused.

To address the limitations, we first introduce a *node group* attribute in the IR graph, where nodes in the same group are fused for execution. Next, we traverse the IR graph using breadth-first search, grouping Limb Pattern nodes into the same group until a node with a different pattern is encountered. This modified dataflow allows for more aggressive fusion across HE operation boundaries. Figure 8 shows the selected dataflow and grouping of nodes in the IR graph. There are five node groups. Within each group, limbs are processed sequentially, and intermediate data is reused directly from the scratchpad memory without any DRAM transfers.

6.2.2 Latency-aware Instruction Scheduling. Our compiler performs instruction scheduling optimizations to reduce stall cycles caused by long DRAM latency. First, we design a prefetching mechanism to minimize the time compute units (i.e., Compute Array and Permute Units) spend waiting for data. During the Instruction Generation pass, we place DMA instructions for the next limb or set of coefficients close to the computation instructions of the current limb, effectively creating a double-buffering scheme to hide DRAM latency. Second, when lowering the IR graph to instructions, we apply a multi-step topological sorting to determine node execution order. We first perform a topological sort to the IR graph by node groups, followed by a topological sort of nodes within each group. This ensures operators within the same group are executed consecutively, without interleaving instructions from other groups. Since nodes within a node group follow a producer-consumer relationship, this scheduling reduces data reuse distance and facilitates efficient reuse through the Scratchpad Memory. For example, as shown in Figure 8, this optimization allows the Automorph in Node Group 5 to be executed closely with the LimbElementwise. In contrast, the processing order based on the standard topological sort would increase data reuse distance, resulting in additional data transfers from DDR memory.

7 Experiments

7.1 Experimental Setup

Platform. We implement the overlay accelerator on an AMD Alveo U280 FPGA. The FPGA has 2 HBM stacks providing a total

DRAM bandwidth of 490 GB/s. There are 1,304K LUTs, 2,607K FFs, 40 MB on-chip SRAM and 9,024 DSPs in the FPGA. The accelerator is developed in Verilog HDL. We use Vivado 2021.1 for synthesis and place-and-route to determine frequency and resource utilization. The accelerator operates at 300 MHz. The reported results are collected after place-and-route. We develop the compiler in Python and run it on a single thread of an AMD Ryzen 3990X CPU.

Benchmarks. We evaluate OLA using the following benchmarks.

- **Basic HE operations.** This benchmark evaluates the latency of basic HE operations across various HE parameters. The HE operations evaluated include Add, Mult, Rotate, Rescale, and Bootstrap.
- **Linear algebra operations.** This benchmark focuses on matrix-matrix multiplications and convolutions, which are commonly used in neural network inference [24].
- **End-to-end HE ML inference.** We evaluate HE ML inference on a single image. We use three CNN models with increasing complexity. CNN-0 [8] is trained on MNIST; CNN-1 [8] and CNN-2 [32] use CIFAR-10. Table 3 lists the details of the CNN models. We consider a common scenario in HE ML inference [24, 32], where the model parameters are in plaintext, and the input images are homomorphically encrypted. We adopt the same ciphertext packing scheme used in prior works [8, 24, 32], which packs elements across multiple channels into a single ciphertext.

Table 3: HE CNN Benchmarks.

Network	CNN-0	CNN-1	CNN-2
Num Layers	6	10	71
Num HE Ops	826	83 K	41 M
Security (λ)	128 bits	192 bits	98 bits
N	2^{13}	2^{14}	2^{16}

Performance Metrics. We evaluate the performance by:

- **Execution latency:** the time from when the first instruction is fetched from DRAM to the time the last instruction completes.
- **Compilation latency:** the time duration from the time the HE ML model (defined using OLA Python DSL) and the input data are provided, to the time the instruction sequence is generated by the compiler.

7.2 Evaluation of HE Operations

We use three sets of HE parameters to evaluate HE operations. The HE parameters are listed in Table 4. These parameters are commonly used in real-world HE applications [8, 23, 41]. Set-1 and Set-2 represent larger HE parameters with more computational levels, while Set-3 uses smaller HE parameters. All sets of parameters meet $\lambda \geq 96$ bits security level.

Table 4: HE Parameters for the evaluation of HE operations.

Parameter Set	N	L	dnum	k	λ
Set-1	2^{16}	44	45	1	98
Set-2	2^{16}	23	3	8	128
Set-3	2^{14}	8	1	1	128

Table 5 shows the execution latency for performing various HE operations on the overlay accelerator. The compilation latency for all HE operations is less than 1 ms. CtCtMult and Rotate, being the most complex HE operations, have significantly higher latency compared to others under the same parameter setting. PtCtAdd and CtCtAdd are memory bound operations with no data reuse, making their latency largely dependent on HBM bandwidth. In contrast, Rescale is compute bound due to the invocations of the Ntt and Intt subroutines.

Table 5: Latency [ms] for executing various HE operations.

Operation	Set-1	Set-2	Set-3
PtCtAdd	0.09	0.05	0.04
CtCtAdd	0.16	0.09	0.08
PtCtMult	2.30	0.96	0.15
CtCtMult	21.67	2.09	0.19
Rescale	2.07	0.84	0.07
Rotate	21.09	2.00	0.18

Unlike prior works [41, 53, 54], our overlay accelerator, referred to as the General Purpose (GP) design, focuses on creating an ISA and hardware architecture to accelerate HE primitives, with the compiler mapping HE operations to these primitives. This approach allows for the reuse of FPGA compute resources (e.g., DSPs) across different HE operations. In contrast, compute units in prior work could be underutilized when HE operations are not executed in parallel. To assess the effectiveness of this approach, we developed an analytical model for a baseline system, called the Fixed Functional (FF) design. Both the GP and FF designs use the same DSP resources. However, in the FF design, half of the DSP resources are allocated for computing Ntt and Intt operations (i.e., NTT Cores), while the rest of the DSPs handle other limb operations [41, 53, 54]. Therefore, NTT cores and the Compute Array (responsible for non-NTT operations) each process 256 coefficients per cycle. Other factors like the scratchpad size, dataflow, and memory bandwidth remain constant. Figure 9 compares latency and utilization of HE operations on the FF and GP designs. The results in the figure are based on HE parameters from Set-3, with similar trends observed for other parameter sets. Excluding memory-bound operations, the GP design consistently demonstrates better DSP utilization, measured by the number of cycles the DSPs are active during execution. This improved utilization leads to up to 1.8× latency reduction.

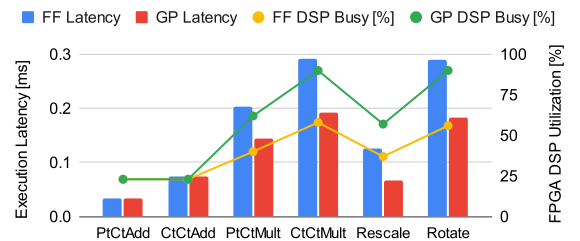


Figure 9: Latency and utilization comparison.

We compare the execution latency of HE operations on the proposed overlay accelerator with state-of-the-art CPU, GPU, and FPGA designs [23, 41, 53]. We use the same HE parameter settings as reported by the prior works. The CPU baseline is measured on a

Table 6: Latency [ms] comparison with CPU, GPU, and FPGA works.

Operation	CPU [23]	GPU [23]	HEAX [41] ¹	Poseidon [53] ¹	Yang [49]	OLA	Speedup over GPU
CtCtAdd	28.12	0.21	0.24	0.08	0.29	0.15	1.40×
PtCtMult	26.22	0.18	0.24	0.08	0.29	0.15	1.20×
CtCtMult	2631.62	17.40	8.40	3.67	12.60	3.92	4.43×
Rescale	144.83	0.64	-	0.25	-	0.39	1.64×
Rotate	2564.11	16.81	-	3.32	11.90	3.75	4.48×
Bootstrap	79444	328.25	-	127.45	-	166.18	1.97×

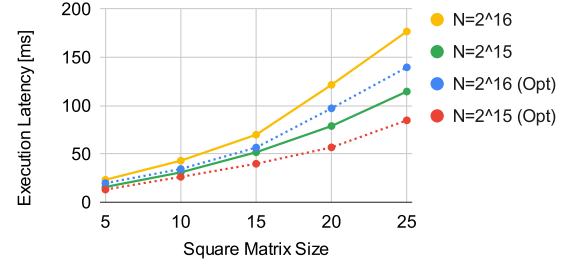
¹ HEAX and Poseidon are optimized for specific HE parameters, whereas OLA supports a broader range of HE parameters.

single thread of an Intel Xeon Gold 6234 CPU. The GPU baseline uses an NVIDIA Tesla V100 GPU. Note that, even though V100 is an older device, it offers more than twice the peak HBM bandwidth of the FPGA platform we use. Table 6 shows the comparison results. CtCtAdd and PtCtMult (excluding Rescale) are simple operations with low computational intensity, therefore the execution time is dominated by the setup cost (e.g., initial instruction and data fetch). OLA achieves only a modest speedup of less than 1.5× compared to the GPU design for these operations. However, our design achieve up to 4.48× speedup over the GPU baseline and over 683× speedup compared to CPU baseline for complex HE operations, which are the bottlenecks in real-world HE applications [23, 53]. HEAX [41] supports only a subset of HE operations, limiting its use to basic HE additions and multiplications. Poseidon [53] is an FPGA accelerator tailored for a *fixed* set of HE parameters. This allows Poseidon to include manual implementation optimizations to achieve high FPGA frequencies (operating at 450 MHz). Yang et al. [49] develop a framework for generating FPGA designs that support all HE operations except Bootstrap, but it requires rerunning the time-consuming FPGA implementation flow when HE parameters change. Our design is the first FPGA-based accelerator that supports a wide range of HE parameters through software programming. Our design outperforms both HEAX and the designs generated by [49]. While Poseidon shows better performance due to its higher frequency, our design is more efficient, requiring up to 1.4× fewer cycles.

7.3 Linear Algebra Operations

We use matrix-matrix multiplication and convolution to demonstrate the effectiveness of our memory bandwidth and access latency aware optimizations. We use HE parameters in Set-2 ($L = 23$) for the evaluation. Figure 10 shows the execution latency of OLA for square matrix-matrix multiplications. We apply diagonal packing for the plaintext matrix and naive packing for the ciphertext matrix, as described in Gazelle [24]. In this scheme, input ciphertext rotations are multiplied by the generalized diagonals of the plaintext matrix and summed to produce the output. We report baseline latency (without operator fusion and latency-aware scheduling) and optimized latency (Opt) with these optimizations enabled. For the matrix-matrix multiplication benchmarks, we observe a minimum improvement of 15%, a maximum improvement of 28%, and a geometric mean improvement of 21%.

Next, we present results for four convolution benchmarks. We use the single input single output with channel packing to encode activation tensors into ciphertext [24, 32]. Convolutions with large

**Figure 10: Latency of matrix multiplication operations.**

input channels increase the required rotations, leading to higher latency. With operator fusion and latency-aware scheduling, OLA consistently outperforms the baseline, achieving up to a 24% reduction in execution latency.

Table 7: Latency [ms] evaluation of convolution operations.

Input (W×H, C)	Filter (W×H, C)	Latency (Baseline)	Latency (Opt)
(28×28, 1)	(5×5, 5)	94.22	84.80
(16×16, 128)	(1×1, 128)	500.06	395.05
(32×32, 32)	(3×3, 32)	142.15	125.09
(16×16, 128)	(3×3, 128)	547.18	415.85

7.4 End-to-end HE ML Inference

The HE CNN inference operations are implemented using the proposed Python DSL. Table 8 shows the comparison results. The CNN benchmarks in the baseline implementations have the same security level (λ) as ours (Table 3). Lola optimizes ciphertext packing for efficient HE CNN inference on multi-core CPUs. Falcon [35] reduces HE operations using homomorphic DFT in convolutional and fully connected layers. Both Lola and Falcon run on an Azure B8ms virtual machine with 8 vCPUs. Lee et al. [32] develop the first HE-optimized ResNet-20 model for the CIFAR-10 dataset and implement it on a multi-core CPU. Compared to the CPU implementations, our design achieves up to 3212× speedup. FxHENN allocates dedicated hardware resources to individual HE subroutines, leading to higher latency due to resource under-utilization. Yang et al. [49] propose a framework for automatically generating FPGA implementations for HE ML inference. OLA outperforms both FxHENN and [49] by up to 108.2×. Poseidon, running at 450 MHz, is a specialized point design for specific FPGA and specific HE parameter. OLA, at 300 MHz, focuses on adaptability, supporting diverse HE parameters and devices. Poseidon achieves lower latency for CNN-2 due to its higher operating frequency (1.5× faster than OLA). However, OLA requires 19% fewer cycles to complete

Table 8: Latency [s] comparison of HE CNN inference.

Network	Work	Platform	$\log Q$	N	Lat. (s)
CNN-0	Lola [8]	8× vCPUs	440	2^{14}	2.2
	Falcon [35]	8× vCPUs	440	2^{14}	1.2
	FxHENN [54]	ACU15EG	210	2^{13}	0.19
	Yang [49]	U280	210	2^{13}	0.02
	OLA	U280	210	2^{13}	0.01
CNN-1	Lola [8]	8× vCPUs	440	2^{14}	730
	Falcon [35]	8× vCPUs	440	2^{14}	107
	FxHENN [54]	ACU15EG	252	2^{14}	54.1
	Yang [49]	U280	252	2^{14}	7.2
	OLA	U280	252	2^{14}	0.5
CNN-2	Lee [32]	Xeon 8280	1750	2^{16}	10602
	Poseidon [53]	U280	960	2^{16}	2.7
	OLA	U280	960	2^{16}	3.3

the inference, demonstrating better efficiency. All existing FPGA designs require running through the time-consuming FPGA implementation flow whenever HE parameters or ML models change. Prior works do not report design implementation times. We observe that it takes 163 minutes to implement our design on the U280 FPGA, and expect other designs to take a similar amount of time. In contrast, our compiler takes less than 1 second to generate the instruction sequence for the CNN models, as shown in Table 9.

Table 9: Compilation latency [ms] of HE CNN models.

	CNN-0	CNN-1	CNN-2
Latency [ms]	67	92	833

7.5 Resource Utilization

Table 10 shows the resource utilization of the overlay accelerator. There are 512 Compute Cells in the Compute Array. Each modular multiplier in the Compute Array requires 12 DSPs, while the modular adders are implemented using LUTs. The capacity of the Scratchpad Memory is 20 MB. The Scratchpad Memory is divided into eight banks, with each bank consisting of 64 URAM and 128 BRAM instances. The Permute Units utilize BRAM resources for their temporal permutation operations.

Table 10: FPGA resource usage of the overlay accelerator.

	DSP	URAM	BRAM	LUT [$\times 10^3$]	FF [$\times 10^3$]
Compute Array	6144	0	0	542	869
Permute Units	0	0	512	109	221
Scratchpad	0	512	1024	17	22
Other Modules	0	0	0	236	426
Total	6144	512	1536	904	1538
[%] of U280	68%	54%	71%	69%	59%

8 Related Works

CPUs/GPUs. Several libraries [3, 19, 24, 46] have been developed to enable efficient execution of HE operations on CPUs. However, CPU implementations of HE still suffer from poor performance due to limited computing resources. Several compilers have been developed to ease the transition of unencrypted applications to HE [11, 13, 14, 33, 36]. All of these efforts target CPU platforms, with

none focusing on custom accelerators. Jung et al. [23] implemented GPU acceleration for various HE operations. Their approach focuses on GPU-specific operation and kernel fusions to reduce latency.

ASICs. CraterLake [44] enables unlimited HE operation by supporting bootstrapping. ARK [27] mitigates the memory bandwidth bottleneck by generating the key switching keys on-the-fly instead of loading them from DRAMs/HBMs. BTS [28] optimizes bootstrapping for a fixed set of HE parameters. While ASIC-based HE accelerators deliver high performance, they require substantial hardware resources, such as hundreds of megabytes of on-chip SRAMs (BTS, ARK).

FPGAs. All the existing FPGA works overlook the software programmable interface needed for the underlying FPGA accelerators. Existing FPGA works can be divided into two categories.

- (1) *FPGA designs with functional limitations.* These works either do not support all HE operations [41, 42, 47, 48, 50] or are limited to fixed HE parameters [1, 2, 53]. FAB [2] designs FPGA accelerator highly optimized for a specific set of HE parameters. Poseidon [53] proposes an FPGA accelerator for HE operations with a high-throughput implementation of Ntt and Automorph. Poseidon is a point design with hard-wired control sequencing, requiring FPGA re-implementation for any changes to HE parameters or HE operations.
- (2) *FPGA frameworks.* FxHENN [54] proposed an HLS-based framework for executing HE operations on FPGAs. Yang et al. [49] developed a framework for automatically generating FPGA designs for different HE operations. Although both frameworks can support various HE operations and parameter settings, they require going through the time-consuming FPGA design flow (synthesis, placement, and routing) whenever the application or HE parameters change.

9 Conclusion

In this paper, we presented OLA, an FPGA-based overlay accelerator for privacy preserving Machine Learning (ML) using Homomorphic Encryption (HE). The hardware architecture is coupled with a flexible Instruction Set Architecture (ISA) to accelerate common HE primitives, while the compiler translates HE programs written in a high-level Python-based DSL into the ISA for execution. OLA incorporates two key optimizations to reduce HE computation latency. First, we introduced an asynchronous dataflow where the compiler manages data processing order based on the computation graph, enabling efficient data reuse through scratchpad memory. Second, we designed latency-aware instruction scheduling to reduce data reuse distance and overlap data transfers with computation. We implemented OLA on an AMD Alveo U280 FPGA. Performance evaluations demonstrate up to 3212× faster than state-of-the-art CPU implementations for various HE ML inference tasks.

10 Acknowledgement

This work has been sponsored by the U.S. National Science Foundation (NSF) under grants SaTC-2104264 and CSSI-2311870. Equipment by AMD AECG is greatly appreciated.

Distribution Statement A: Approved for public release. Distribution is unlimited.

References

- [1] Rashmi Agrawal, Anantha Chandrakasan, and Ajay Joshi. 2024. HEAP: A Fully Homomorphic Encryption Accelerator with Parallelized Bootstrapping. In *2024 ACM/IEEE 51st International Symposium on Computer Architecture (ISCA)*.
- [2] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. FAB: An FPGA-based accelerator for bootstrappable fully homomorphic encryption. In *2023 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. *arXiv preprint arXiv:2207.11872*.
- [3] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, et al. 2022. OpenFHE: Open-source fully homomorphic encryption library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 53–63.
- [4] Martin Albrecht, Melissa Chase, Hao Chen, and et al. 2018. *Homomorphic Encryption Security Standard*. Technical Report.
- [5] David Archer, Lily Chen, Jung Hee Cheon, Ran Gilad-Bachrach, Roger A Hallman, Zhicong Huang, Xiaoqian Jiang, Ranjit Kumaresan, Bradley A Malin, Heidi Sofia, et al. 2017. Applications of homomorphic encryption. In *Crypto Standardization Workshop, Microsoft Research*, Vol. 14. sn.
- [6] Jean-Claude Bajard, Julien Eynard, M. Anwar Hasan, and Vincent Zucca. 2017. A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes. In *Selected Areas in Cryptography – SAC 2016*, Roberto Avanzi and Howard Heys (Eds.).
- [7] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2012. (Leveled) Fully Homomorphic Encryption without Bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*.
- [8] Alon Brutzkus, Ran Gilad Bachrach, and Oren Elisha. 2019. Low Latency Privacy Preserving Inference. In *Proceedings of the 36th International Conference on Machine Learning*.
- [9] R. Chen and V. K. Prasanna. 2015. Automatic generation of high throughput energy efficient streaming architectures for arbitrary fixed permutations. In *2015 25th International Conference on Field Programmable Logic and Applications (FPL)*, 1–8.
- [10] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2019. A Full RNS Variant of Approximate Homomorphic Encryption. In *Selected Areas in Cryptography – SAC 2018*, Carlos Cid and Michael J. Jacobson Jr. (Eds.). Springer International Publishing, Cham, 347–368.
- [11] Sangeeta Chowdhary, Wei Dai, Kim Laine, and Olli Saarikivi. 2021. Eva improved: Compiler and extension library for ckks. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 43–55.
- [12] Jason Cong, Peng Wei, Cody Hao Yu, and Peng Zhang. 2018. Automated Accelerator Generation and Optimization with Composable, Parallel and Pipeline Architecture (DAC '18). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3195970.3195999>
- [13] Roshan Dathathri, Blagovesta Kostova, Olli Saarikivi, Wei Dai, Kim Laine, and Madan Musuvathi. 2020. EVA: an encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 546–561.
- [14] Roshan Dathathri, Olli Saarikivi, Hao Chen, Kim Laine, Kristin Lauter, Saeed Maleki, Madanlal Musuvathi, and Todd Mytkowicz. 2019. CHET: an optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 142–156.
- [15] Leo de Castro, Rashmi Agrawal, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, Chiraag Juvekar, and Ajay Joshi. 2021. Does Fully Homomorphic Encryption Need Compute Acceleration? *arXiv preprint arXiv:2112.06396* (2021).
- [16] Nathan Dowlin, Ran Gilad-Bachrach, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. [n. d.]. CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy (ICML '16).
- [17] Zekeriya Erkin, Thijs Veugen, Tomas Toft, and Reginald L. Lagendijk. 2012. Generating private recommendations efficiently using homomorphic encryption and data packing. *IEEE transactions on information forensics and security* 7, 3 (2012), 1053–1066.
- [18] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*. Association for Computing Machinery.
- [19] Shai Halevi and Victor Shoup. 2020. Design and implementation of HELib: a homomorphic encryption library. *Cryptology ePrint Archive* (2020).
- [20] Kyoohyung Han and Dohyeong Ki. 2020. Better bootstrapping for approximate homomorphic encryption. In *Cryptographers' Track at the RSA Conference*. Springer, 364–390.
- [21] Darrel Hankerson, Alfred J. Menezes, and Scott Vanstone. 2003. *Guide to Elliptic Curve Cryptography*. Springer-Verlag, Berlin, Heidelberg.
- [22] John Hennessy and David Patterson. 2018. A new golden age for computer architecture: domain-specific hardware/software co-design, enhanced. In *ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*.
- [23] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2021), 114–148.
- [24] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. 2018. {GAZELLE}: A low latency framework for secure neural network inference. In *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 1651–1669.
- [25] Jinsu Kim, Dongyoung Koo, Yuna Kim, Hyunsoo Yoon, Junbum Shin, and Sungwook Kim. 2018. Efficient privacy-preserving matrix factorization for recommendation via fully homomorphic encryption. *ACM Transactions on Privacy and Security (TOPS)* 21, 4 (2018), 1–30.
- [26] Junsoo Kim, Chanhwa Lee, Hyungbo Shim, Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2016. Encrypting controller using fully homomorphic encryption for security of cyber-physical systems. *IFAC-PapersOnLine* 49, 22 (2016), 175–180.
- [27] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1237–1254.
- [28] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2021. BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption. *arXiv preprint arXiv:2112.15479* (2021).
- [29] Sunwoong Kim, Keewoo Lee, Wonhee Cho, Jung Hee Cheon, and Rob A Rutenbar. 2019. FPGA-based accelerators of fully pipelined modular multipliers for homomorphic encryption. In *2019 International Conference on ReConfigurable Computing and FPGAs (ReConFig)*. IEEE, 1–8.
- [30] Övünç Kocabaş and Tolga Soyata. 2016. Medical data analytics in the cloud using homomorphic encryption. In *E-Health and Telemedicine: Concepts, Methodologies, Tools, and Applications*. IGI Global, 751–768.
- [31] Övünç Kocabaş and Tolga Soyata. 2020. Towards privacy-preserving medical cloud computing using homomorphic encryption. In *Virtual and Mobile Healthcare: Breakthroughs in Research and Practice*. IGI Global, 93–125.
- [32] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. 2022. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access* 10 (2022), 30039–30054.
- [33] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. 2024. Performance-aware scale analysis with reserve for homomorphic encryption. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 302–317.
- [34] Yihao Liu, Patrick Hung, Ehsan Nekouei, and Ray CC Cheung. 2023. Homomorphic Encryption-Based System Design for Secure Data Processing. In *2023 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC)*. IEEE, 1–6.
- [35] Qian Lou, Wen-jie Lu, Cheng Hong, and Lei Jiang. 2020. Falcon: Fast spectral inference on encrypted data. *Advances in Neural Information Processing Systems* 33 (2020), 2364–2374.
- [36] Raghav Malik, Kabir Sheth, and Milind Kulkarni. 2023. Coyote: A compiler for vectorizing encrypted arithmetic circuits. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 118–133.
- [37] Oliver Masters, Hamish Hunt, Enrico Steffanlongo, Jack Crawford, Flavio Bergamaschi, Maria E Dela Rosa, Caio C Quini, Camila T Alves, Fernanda de Souza, and Deise G Ferreira. 2019. Towards a homomorphic machine learning big data pipeline for the financial services sector. *Cryptology ePrint Archive* (2019).
- [38] Razvan Nane and et al. 2016. A Survey and Evaluation of FPGA High-Level Synthesis Tools. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2016).
- [39] Hongwu Peng, Ran Ran, Yukui Luo, Jiahui Zhao, Shaoyi Huang, Kiran Thorat, Tong Geng, Chenghong Wang, Xiaolin Xu, Wujie Wen, et al. 2024. Lingn: Structural linearized graph convolutional network for homomorphically encrypted inference. *Advances in Neural Information Processing Systems* 36 (2024).
- [40] Sathi Sarveswara Reddy, Sharad Sinha, and Wei Zhang. 2023. Design and Analysis of RSA and Paillier Homomorphic Cryptosystems Using PSO-Based Evolutionary Computation. *IEEE Trans. Comput.* 72, 7 (2023), 1886–1900.
- [41] M. Sadegh Riazzi, Kim Laine, Blake Pelton, and Wei Dai. [n. d.]. HEAX: An Architecture for Computing on Encrypted Data (ASPLOS '20).
- [42] Sujoy Sinha Roy, Ahmet Can Mert, Sunmin Kwon, Youngsam Shin, Donghoon Yoo, et al. 2021. Accelerator for Computing on Encrypted Data. *Cryptology ePrint Archive* (2021).
- [43] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. *F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption*. Association for Computing Machinery, New York, NY, USA.
- [44] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel

- Sanchez. 2022. Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 173–187.
- [45] Vivek Sarkar. 1988. Synchronization using counting semaphores. In *Proceedings of the 2nd International Conference on Supercomputing*. 627–637.
- [46] SEAL 2020. Microsoft SEAL (release 3.6). <https://github.com/Microsoft/SEAL>. Microsoft Research, Redmond, WA..
- [47] S. Sinha Roy, F. Turan, K. Jarvinen, F. Vercauteren, and I. Verbauwhede. 2019. FPGA-Based High-Performance Parallel Architecture for Homomorphic Computing on Encrypted Data. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*.
- [48] Zhihan Xu, Yang Yang, Rajgopal Kannan, and Viktor K Prasanna. 2024. Bandwidth Efficient Homomorphic Encrypted Discrete Fourier Transform Acceleration on FPGA. In *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 1–12.
- [49] Yang Yang, Rajgopal Kannan, and Viktor Prasanna. 2024. A Framework for Generating Accelerators for Homomorphic Encryption Operations on FPGAs. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE.
- [50] Yang Yang, Sanmukh R Kuppannagari, Rajgopal Kannan, and Viktor K Prasanna. 2022. Bandwidth Efficient Homomorphic Encrypted Matrix Vector Multiplication Accelerator on FPGA. In *2022 IEEE International Conference Field-Programmable Technology (FPT)*. IEEE.
- [51] Yang Yang, Sanmukh R Kuppannagari, Rajgopal Kannan, and Viktor K Prasanna. 2022. NTTGen: a framework for generating low latency NTT implementations on FPGA. In *Proceedings of the 19th ACM International Conference on Computing Frontiers*. 30–39.
- [52] Yang Yang, Weihang Long, Rajgopal Kannan, and Viktor K Prasanna. 2023. FPGA Acceleration of Rotation in Homomorphic Encryption using Dynamic Data Layout. In *2023 IEEE Annual International Symposium on Field-Programmable Logic and Applications (FPL)*. IEEE.
- [53] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical Homomorphic Encryption Accelerator. In *2023 IEEE International Symposium on High Performance Computer Architecture (HPCA)*.
- [54] Yilan Zhu, Xinyao Wang, Lei Ju, and Shanqing Guo. 2023. FxHENN: FPGA-based acceleration framework for homomorphic encrypted CNN inference. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 896–907. <https://doi.org/10.1109/HPCA56546.2023.10071133>